

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Luis Gustavo Bonfim Ciaramicoli

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	3
3	Quadro de entregas	4
4	Atividades realizadas	5
4.1	Atividade 1 — Agenda telefônica em Flask	5
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	7
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	9
4.4	Atividade 4 — Controle de acesso por papel — RBAC	11
4.5	Atividade 5 — Logs e auditoria	13
4.6	Atividade 6 — Upload e perfil de usuário	15
5	Atividades extras (opcional)	17
5.1	Atividade E1 — Documentação Swagger/OpenAPI	17
5.2	Atividade E2 — CI/CD com GitHub Actions	19
5.3	Atividade E3 — Observabilidade — health checks e métricas	21
6	Considerações finais	23
7	Declaração de autoria	23

1 Introdução

A disciplina Introdução à Computação em Nuvem (ISW055) aborda a transição de softwares monolíticos para arquiteturas modulares, distribuídas e containerizadas, executadas sobre infraestruturas escaláveis e compartilhadas. Durante o primeiro bimestre do semestre letivo de 2026.2, o conteúdo programático abrangeu conceitos fundamentais de computação em nuvem, destacando a containerização com Docker, a orquestração local de múltiplos serviços com Docker Compose, o padrão arquitetural de microsserviços, o controle de acesso baseado em papéis (RBAC), a mensageria e auditoria assíncrona com Redis Streams, o armazenamento de objetos com MinIO compatível com S3 e a observabilidade com Prometheus e Grafana.

Como eixo prático das aulas e entregas avaliativas, foi desenvolvido progressivamente um catálogo de filmes dedicado à filmografia do ator Tom Hanks, consumindo a API pública do The Movie Database (TMDB). A aplicação evoluiu incrementalmente de um monólito inicial para uma arquitetura distribuída composta por gateway reverso, microsserviço isolado de autenticação (auth-service), microsserviço de auditoria (log-service), banco relacional MariaDB, mensageria com Redis e armazenamento de arquivos no MinIO. Ao término do ciclo bimestral, toda a infraestrutura foi implantada e validada na instância institucional do Portainer da Fatec Pompeia na porta 8218.

2 Metodologia

O desenvolvimento prático das atividades distribuiu-se em dois repositórios públicos no GitHub: as entregas iniciais de nivelamento e a versão preliminar do catálogo foram implementadas no repositório <https://github.com/luiscaramicoli/cloude>, enquanto o desacoplamento em microsserviços, o provisionamento de bancos de dados, os fluxos assíncronos e os recursos extras foram consolidados no repositório <https://github.com/luiscaramicoli/cloud-atividade>. O ambiente de desenvolvimento utilizou o sistema operacional Linux (Fedora Workstation) e o editor Visual Studio Code. A pilha técnica reuniu Python com Flask no módulo inicial de nivelamento, evoluindo para Node.js 20 LTS com Express 5 nos serviços principais, MariaDB para persistência relacional, Redis 7 para mensageria de eventos, MinIO RELEASE.2025 para armazenamento de objetos e Docker Compose v2 para padronização do ambiente local.

A homologação e entrega do ambiente ocorreram no cluster do Portainer da Fatec Pompeia sob a porta 8218. Em decorrência de restrições de privilégios para montagem de volumes locais (bind-mounts) e dos recursos limitados para compilação remota no servidor compartilhado, o processo de entrega contínua foi estruturado por meio de uma pipeline no GitHub Actions. O fluxo automatizado executa verificações de integridade, compila as imagens dos microsserviços, publica os pacotes no GitHub Container Registry (GHCN) e aciona o webhook do Portainer para atualização automática da stack em produção. A validação funcional das rotas e das políticas de segurança foi conduzida via navegador web e requisições HTTP instrumentadas com `curl` e clientes REST.

As datas e horários reportados nas fichas de entrega foram extraídos diretamente do histórico de commits do Git nos repositórios remotos, obtidos por meio de consultas `git log` no terminal e validados na interface do GitHub, com conversão sistemática para o fuso horário oficial de Brasília (UTC-3). Como apoio consultivo ao longo do desenvolvimento, foram empregados modelos de inteligência artificial generativa (Claude e Gemini) para assistência em rotinas repetitivas de código e diagnóstico de falhas, mantendo-se toda a formulação lógica, integração arquitetural, testes e configuração de infraestrutura sob a autoria, revisão e validação direta do aluno.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	14/08/2026 19:59	entregue com atraso
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 15:18	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	27/08/2026 00:27	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	03/09/2026 15:58	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	25/09/2026 14:14	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	30/09/2026 14:16	entregue
E1	Documentação Swagger/OpenAPI	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.	sem prazo	18/09/2026 14:34	entregue
E2	CI/CD com GitHub Actions	Pipeline que testa e faz deploy a cada push.	sem prazo	18/09/2026 17:31	entregue
E3	Observabilidade — health checks e métricas	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.	sem prazo	18/09/2026 17:31	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	14/08/2026 19:59
Situação	entregue com atraso
Evidência	GitHub — commits a6e2cce (versão com JSON) e 1eb4711 (CRUD + Docker) no repositório cloude
Link	https://github.com/luiscaramicoli/cloude/commit/a6e2ccea64df6605f8d117c102985b2756e8f68d

Sobre a data e justificativa do status. A atividade prática de nivelamento foi desenvolvida em sala de aula na data prevista (07/08/2026). No entanto, o envio inicial para o repositório público no GitHub ocorreu somente em 14/08/2026 às 19:59. Como a comprovação formal exige um registro público verificável, classifiquei a entrega no quadro como entregue com atraso.

O que foi feito. Desenvolvi uma aplicação monolítica em Python com Flask (app.py), usando Jinja inline (render_template_string) para a página web de cadastro e listagem de contatos (nome, telefone e valor), além de rotas de API em formato JSON (POST /api/registro e GET /api/registros). Na primeira versão, os dados eram salvos no arquivo local dados.json (a6e2cce). Na sequência, atualizei o código para um CRUD completo com Flask-SQLAlchemy, aceitando conexão dinâmica com MySQL ou MariaDB via variáveis de ambiente com fallback para SQLite local em banco_local.db, e criei um Dockerfile baseado em python:3.12-slim (1eb4711, 14/08/2026 20:36). Mais tarde, esses arquivos foram substituídos pelo catálogo no mesmo repositório, mas permanecem disponíveis no histórico do Git (árvore do commit histórico 1eb4711).

https://github.com/luiscaramicoli/cloude/commit/a6e2ccea64df6605f8d117c102985b2756e8f68d

Platform Solutions Resources Open Source Enterprise Pricing

luiscaramicoli / cloude

<> Code Issues 1 Pull requests Actions Projects Security and quality Insights

Commit a6e2cce

luiscaramicoli committed on Aug 14

Commit inicial

main 0 parents commit a6e2cce

2 files changed +310

```

"sha": "a6e2ccea64df6605f8d117c102985b2756e8f68d"
"commit.author": "luiscaramicoli" "2026-08-14T22:59:55Z" -> 14/08/2026, 19:59:55 (horário de Brasília)
"commit.committer": "luiscaramicoli" "2026-08-14T22:59:55Z" -> 14/08/2026, 19:59:55 (horário de Brasília)
"commit.message": "Commit inicial"

```

Figura 1: Atividade 1 — commit a6e2cce no GitHub — página do commit e data/hora exata pela API do GitHub

http://127.0.0.1:1753/ capturado em 06/10/2026, 16:38:45

Cadastrar Novo Registro

Nome Completo
Ex: João da Silva

Telefone
Ex: (11) 99999-9999

Valor (R\$)
Ex: 150.50

Cadastrar

Lista de Registros

Luis Ciaramicoli 📞 11999999999	R\$ 150.50
Luis Bonfim 📞 11555555555	R\$ 700.50

Figura 2: Atividade 1 — agenda telefônica em execução no navegador

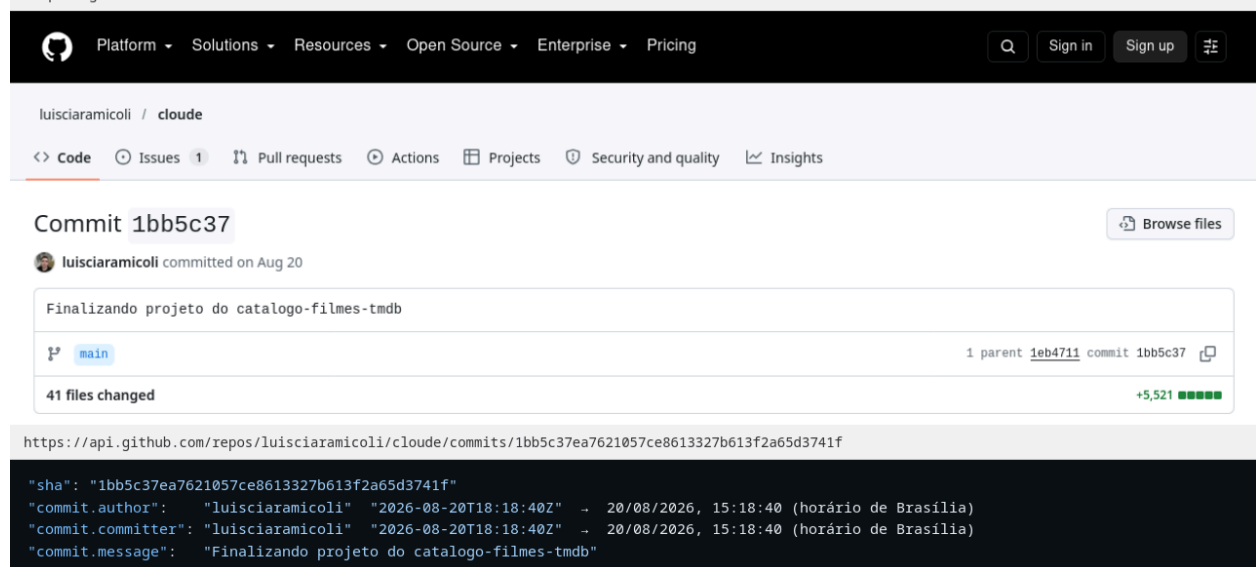
Dificuldades e como foram resolvidas. A maior dificuldade foi deixar a conexão do banco flexível para rodar tanto localmente quanto em container. Resolvi isso lendo as variáveis de ambiente no arquivo `.env`, conectando no MySQL quando as configurações existiam ou mantendo o SQLite por padrão caso contrário. Também precisei mapear a porta de exposição no contêiner Docker e assegurar que as dependências do `requirements.txt` fossem instaladas sem gerar imagens excessivamente volumosas.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 15:18
Situação	entregue
Evidência	GitHub — commit 1bb5c37 + README + print do catálogo (repositório cloude)
Link	https://github.com/luiscaramicoli/cloude/commit/1bb5c37ea7621057ce8613327b613f2a65d3741f

O que foi feito. Criei uma aplicação em Node.js com Express (pasta `catalogo-filmes-tmdb`) que consome a API do TMDB para listar e pesquisar filmes do ator Tom Hanks. O banco de dados MariaDB foi estruturado para separar os dados por usuário, permitindo que cada conta favoritasse filmes e salvasse seus próprios comentários. As senhas dos usuários foram protegidas com hash usando `bcrypt` e a sessão foi gerenciada por tokens JWT. A aplicação foi empacotada com `Dockerfile` e `docker-compose.yml` para rodar no Portainer, e o arquivo `README.md` inclui as instruções e a menção ao professor Allan Siriani (@siriani).

<https://github.com/luiscaramicoli/cloude/commit/1bb5c37ea7621057ce8613327b613f2a65d3741f>



```

"sha": "1bb5c37ea7621057ce8613327b613f2a65d3741f"
"commit.author": "luiscaramicoli" "2026-08-20T18:18:40Z" -> 20/08/2026, 15:18:40 (horário de Brasília)
"commit.committer": "luiscaramicoli" "2026-08-20T18:18:40Z" -> 20/08/2026, 15:18:40 (horário de Brasília)
"commit.message": "Finalizando projeto do catalogo-filmes-tmdb"

```

Figura 3: Atividade 2 — commit 1bb5c37 no GitHub — página do commit e data/hora exata pela API do GitHub

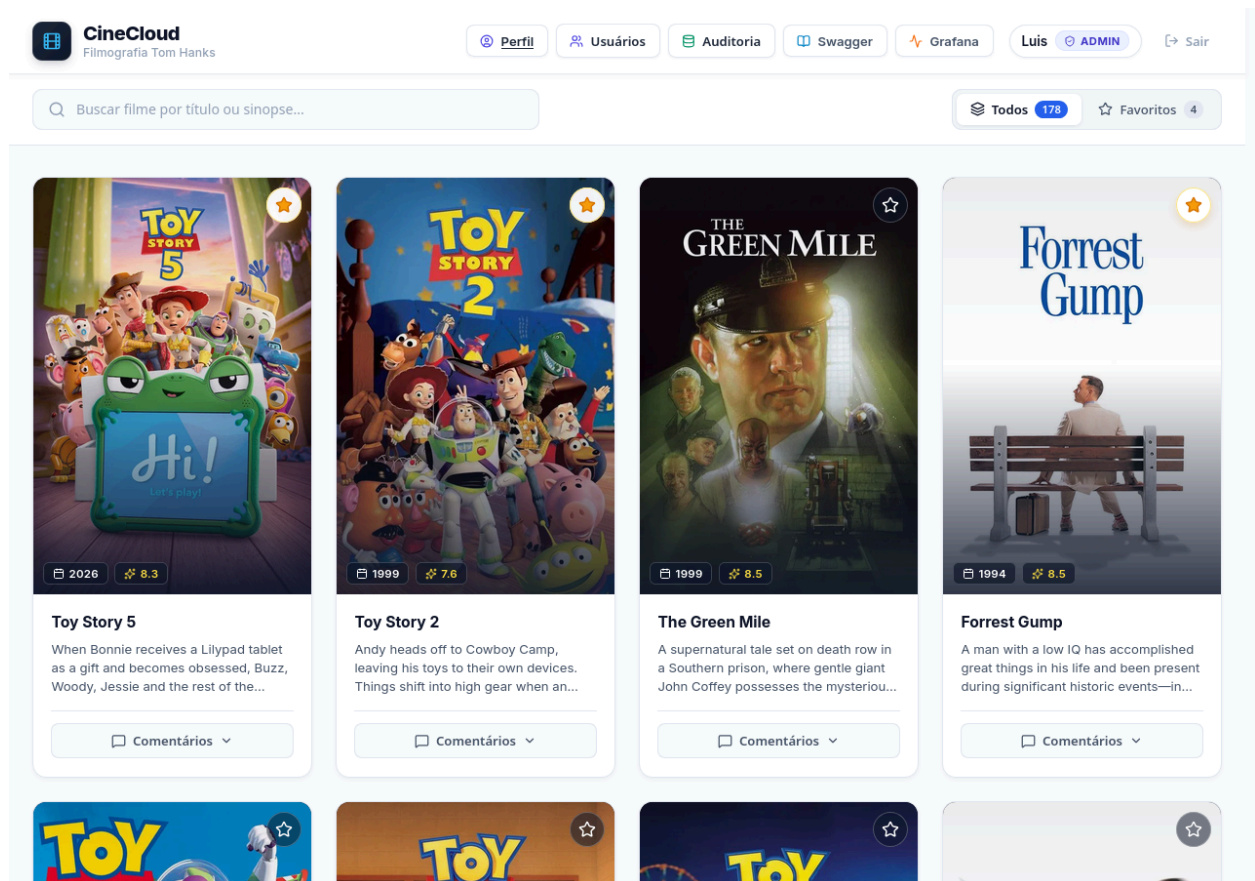


Figura 4: Atividade 2 — catálogo de filmes em execução consumindo API TMDb

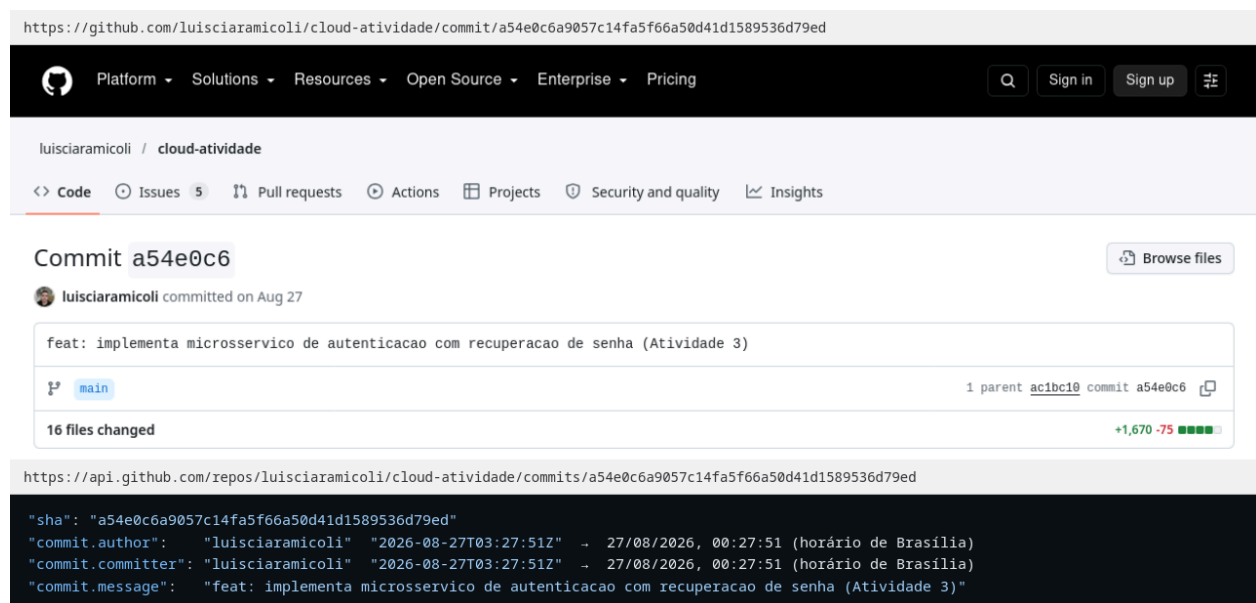
Dificuldades e como foram resolvidas. Após o commit inicial, precisei parametrizar a porta externa no `docker-compose.yml` (577b26d) para evitar conflito com outros serviços no Portainer. No frontend, ajustei as regras de CSS para garantir que a página ficasse responsiva no celular (6e7d715) e limpei o cache do navegador para atualizar os estilos. Também corrigi o README, que mencionava React por engano, deixando registrado que a interface foi feita em HTML, CSS e JavaScript puro (1ee1de3).

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	27/08/2026 00:27
Situação	entregue
Evidência	GitHub — commit a54e0c6 + docker-compose.yml + print do login
Link	https://github.com/luiscaramicoli/cloud-atividade/commit/a54e0c6a9057c14fa5f66a50d41d1589536d79ed

O que foi feito. Separei a parte de segurança do restante da aplicação, movendo as rotas de login, cadastro e recuperação de senha para um microsserviço independente chamado auth-service. Esse serviço roda em uma rede interna e isolada no Docker (backend-network), sem expor portas para o host externo. O backend do catálogo passou a atuar como gateway e proxy reverso, recebendo as requisições do frontend e repassando as chamadas de autenticação internamente. Depois dessa separação, reorganizei o projeto no padrão MVC (80abdbe), dividindo rotas, controladores, serviços e configurações.

<https://github.com/luiscaramicoli/cloud-atividade/commit/a54e0c6a9057c14fa5f66a50d41d1589536d79ed>



```

"sha": "a54e0c6a9057c14fa5f66a50d41d1589536d79ed"
"commit.author": "luiscaramicoli" "2026-08-27T03:27:51Z" → 27/08/2026, 00:27:51 (horário de Brasília)
"commit.committer": "luiscaramicoli" "2026-08-27T03:27:51Z" → 27/08/2026, 00:27:51 (horário de Brasília)
"commit.message": "feat: implementa microsservico de autenticao com recuperacao de senha (Atividade 3)"

```

Figura 5: Atividade 3 — commit a54e0c6 no GitHub — página do commit e data/hora exata pela API do GitHub

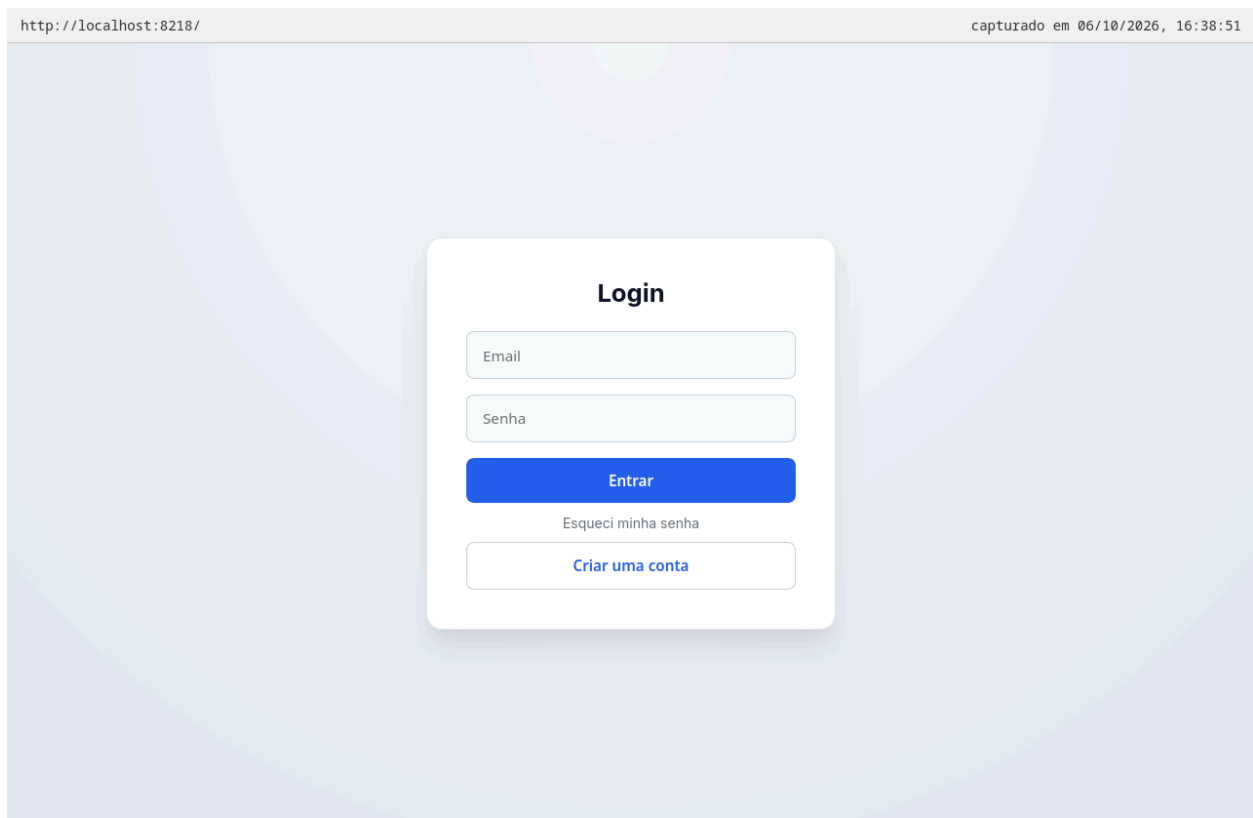


Figura 6: Atividade 3 — tela de autenticação e recuperação de senha pelo microserviço

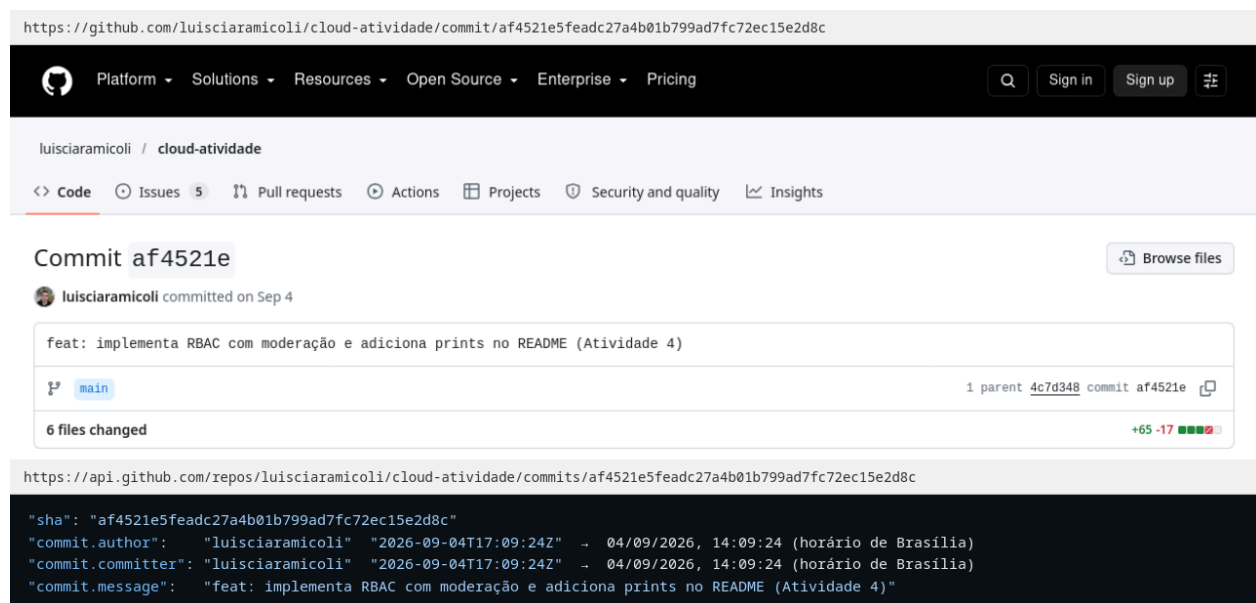
Dificuldades e como foram resolvidas. Ao rodar dois serviços Node.js conectados pela rede interna do Docker, tive problemas com a resolução de nomes dos containers e o tempo que o banco demorava para iniciar. Resolvi configurando a variável `AUTH_SERVICE_URL=http://auth-service:5001` no gateway e adicionando uma rotina com repetições automáticas para esperar o MariaDB aceitar conexões antes de subir a aplicação.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	03/09/2026 15:58
Situação	entregue
Evidência	GitHub — commit 652be0c + prints 403 e 200 de admin
Link	https://github.com/luiscaramicoli/cloud-atividade/commit/652be0cb29ffaa61daec21dbec81643c72d6dcfa

O que foi feito. Implementei controle de acesso baseado em papéis (RBAC) com verificação rigorosa no backend. Adicionei a coluna role (usuario ou admin) na tabela de usuários e criei a regra de moderação: um usuário comum só pode apagar os seus próprios comentários (`comment.usuario_id === req.userId`), enquanto o administrador tem permissão para apagar comentários de qualquer pessoa. Para evitar adulteração, o backend consulta o papel diretamente no banco de dados a cada requisição sensível, em vez de confiar apenas nas informações presentes no token JWT.

<https://github.com/luiscaramicoli/cloud-atividade/commit/af4521e5feadc27a4b01b799ad7fc72ec15e2d8c>



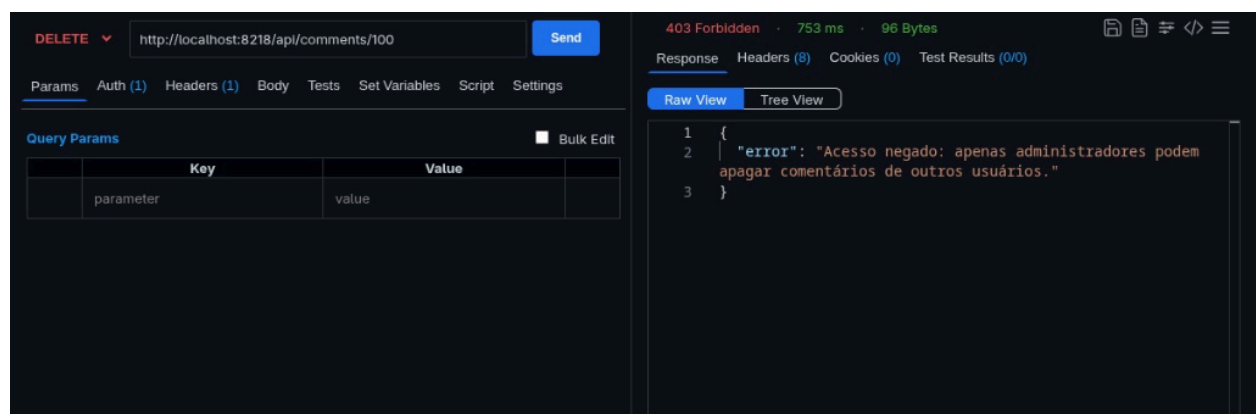
<https://api.github.com/repos/luiscaramicoli/cloud-atividade/commits/af4521e5feadc27a4b01b799ad7fc72ec15e2d8c>

```

"sha": "af4521e5feadc27a4b01b799ad7fc72ec15e2d8c"
"commit.author": "luiscaramicoli" "2026-09-04T17:09:24Z" → 04/09/2026, 14:09:24 (horário de Brasília)
"commit.committer": "luiscaramicoli" "2026-09-04T17:09:24Z" → 04/09/2026, 14:09:24 (horário de Brasília)
"commit.message": "feat: implementa RBAC com moderação e adiciona prints no README (Atividade 4)"

```

Figura 7: Atividade 4 — commit 652be0c no GitHub — página do commit e data/hora exata pela API do GitHub



DELETE <http://localhost:8218/api/comments/100> **Send**

Params Auth (1) Headers (1) Body Tests Set Variables Script Settings

Query Params ☐ Bulk Edit

Key	Value
parameter	value

Response **403 Forbidden** · 753 ms · 96 Bytes

Response Headers (8) Cookies (0) Test Results (0/0)

Raw View **Tree View**

```

1 {
2   "error": "Acesso negado: apenas administradores podem
3     apagar comentários de outros usuários."

```

Figura 8: Atividade 4 — tentativa de exclusão por usuário comum negada com 403 Forbidden

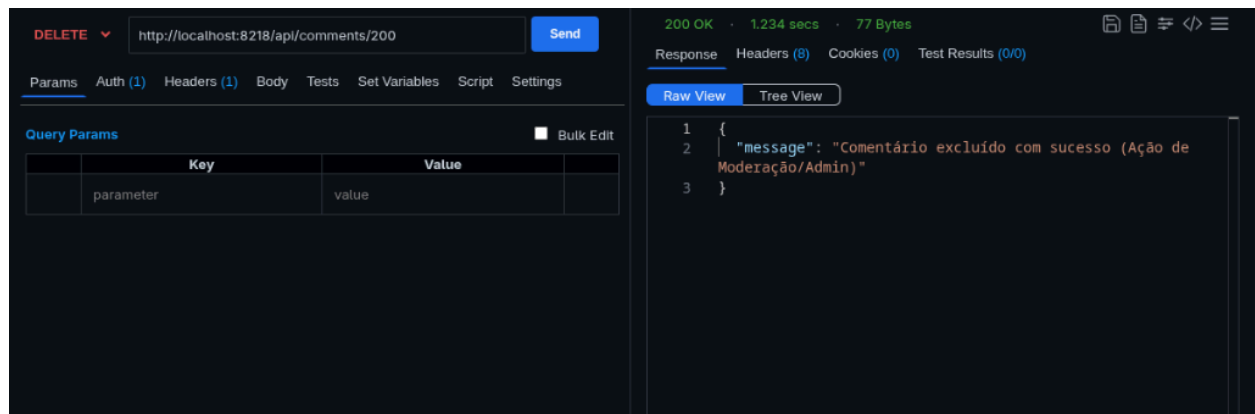


Figura 9: Atividade 4 — moderação executada com sucesso por usuário administrador (200 OK)

Dificuldades e como foram resolvidas. Como as tabelas já tinham registros criados nas semanas anteriores, precisei criar uma migration segura com `ALTER TABLE usuarios ADD COLUMN role ...` que definisse o papel usuario por padrão para não quebrar os logins existentes, além de um script para promover manualmente a primeira conta a admin. Também corriji o retorno dos erros, padronizando a resposta 403 Forbidden no Express em vez de 401 Unauthorized quando a credencial era válida, mas a conta não tinha privilégio suficiente.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	25/09/2026 14:14
Situação	entregue
Evidência	GitHub — commit 6ff06b9 + prints dos logs e painel de auditoria
Link	https://github.com/luisciaramicoli/cloud-atividade/commit/6ff06b9ba1d8e1c6b12a2333b23e1ca313364f34

O que foi feito. Criei um novo microserviço (log-service) dedicado ao registro e consulta de eventos de auditoria, integrado ao Redis Streams. Os eventos seguem o padrão estruturado CloudEvents v1.0 e as diretrizes do NIST SP 800-92, registrando timestamp UTC, identificador do usuário, endereço IP de origem, ação realizada e resultado da operação. O catálogo e o auth-service despacham os logs de forma assíncrona para que a gravação não deixe as respostas aos usuários mais lentas. Também construí uma tela restrita a administradores para filtrar os logs por tipo de ação, período e nível de severidade.

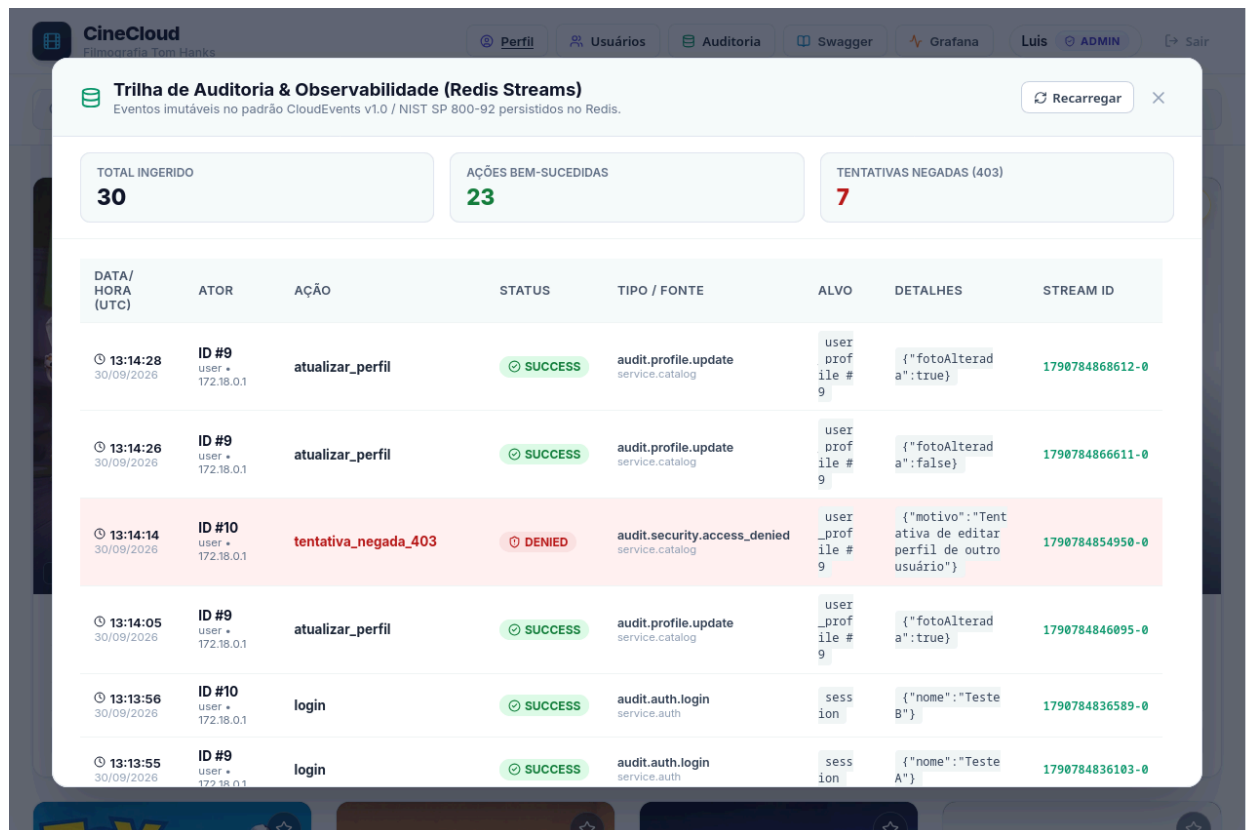
<https://github.com/luisciaramicoli/cloud-atividade/commit/c18479e9f0f3ddcd5acde7572e93e47388c982d0>

```

"sha": "c18479e9f0f3ddcd5acde7572e93e47388c982d0"
"commit.author": "luisciaramicoli" "2026-09-18T16:49:10Z" → 18/09/2026, 13:49:10 (horário de Brasília)
"commit.committer": "luisciaramicoli" "2026-09-18T16:49:10Z" → 18/09/2026, 13:49:10 (horário de Brasília)
"commit.message": "feat: implementa observabilidade com microserviço de logs e auditoria em redis streams (Atividade 5)"

```

Figura 10: Atividade 5 — commit 6ff06b9 no GitHub — página do commit e data/hora exata pela API do GitHub



DATA/ HORA (UTC)	ATOR	AÇÃO	STATUS	TIPO / FONTE	ALVO	DETALHES	STREAM ID
🕒 13:14:28 30/09/2026	ID #9 user • 172.18.0.1	atualizar_perfil	🟢 SUCCESS	audit.profile.update service.catalog	user _profil e # 9	{"fotoAlterad a":true}	1790784868612-0
🕒 13:14:26 30/09/2026	ID #9 user • 172.18.0.1	atualizar_perfil	🟢 SUCCESS	audit.profile.update service.catalog	user _profil e # 9	{"fotoAlterad a":false}	1790784866611-0
🕒 13:14:14 30/09/2026	ID #10 user • 172.18.0.1	tentativa_negada_403	🔴 DENIED	audit.security.access_denied service.catalog	user _profil e # 9	{"motivo":"Tent ativa de editar perfil de outro usuário"}	1790784854950-0
🕒 13:14:05 30/09/2026	ID #9 user • 172.18.0.1	atualizar_perfil	🟢 SUCCESS	audit.profile.update service.catalog	user _profil e # 9	{"fotoAlterad a":true}	1790784846095-0
🕒 13:13:56 30/09/2026	ID #10 user • 172.18.0.1	login	🟢 SUCCESS	audit.auth.login service.auth	sess ion	{"nome":"Teste B"}	1790784836589-0
🕒 13:13:55 30/09/2026	ID #9 user • 172.18.0.1	login	🟢 SUCCESS	audit.auth.login service.auth	sess ion	{"nome":"Teste A"}	1790784836103-0

Figura 11: Atividade 5 — consulta de logs de auditoria pelo administrador com paginação e filtros

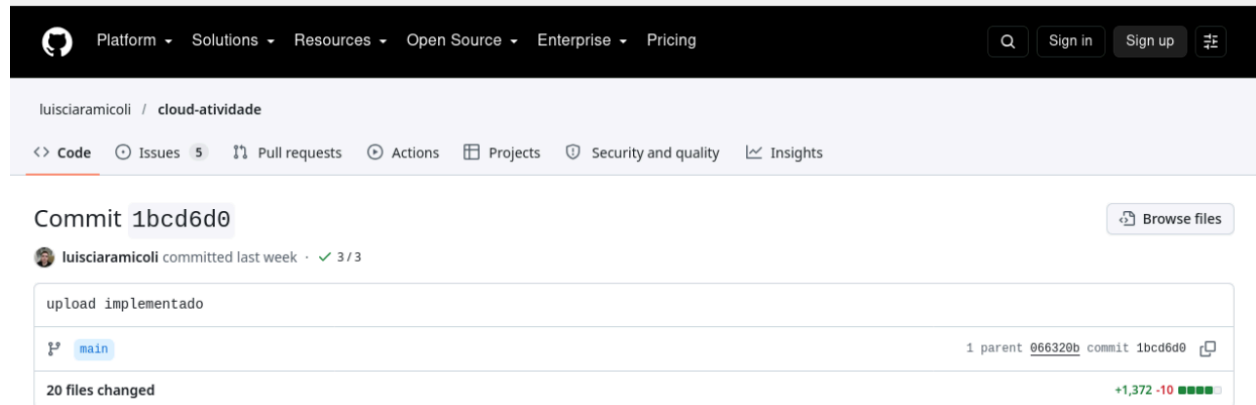
Dificuldades e como foram resolvidas. A principal dificuldade foi garantir a continuidade da aplicação caso o Redis ficasse temporariamente fora do ar. Implementei um padrão resiliente com timeout curto e tratamento de erro silencioso no envio do log: caso o Redis falhe, o erro é registrado no console do container, mas a operação do usuário (como login ou comentário) é concluída normalmente sem travar o sistema.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	30/09/2026 14:16
Situação	entregue
Evidência	GitHub — commit ffa84d9 + prints de upload e avatar no MinIO
Link	https://github.com/luisciaramicoli/cloud-atividade/commit/ffa84d9dbd00465c026caee4a3ec687799ea7da8

O que foi feito. Implementei a funcionalidade de perfil com envio de foto de avatar, integrando o serviço de armazenamento de objetos MinIO (compatível com a API AWS S3). Apenas a URL do objeto é salva no banco de dados MariaDB; o binário da imagem é armazenado diretamente no bucket do MinIO. O upload utiliza multer no backend com validação de tipo de arquivo (permitindo apenas JPG, PNG e WebP) e limite de tamanho de 2 MB. O controle de acesso impede que um usuário envie imagens para o perfil de outra pessoa.

<https://github.com/luisciaramicoli/cloud-atividade/commit/1bcd6d06fed52629ec9361174af182b657149111>



<https://api.github.com/repos/luisciaramicoli/cloud-atividade/commits/1bcd6d06fed52629ec9361174af182b657149111>

```

"sha": "1bcd6d06fed52629ec9361174af182b657149111"
"commit.author": "luisciaramicoli" "2026-09-30T16:20:05Z" -> 30/09/2026, 13:20:05 (horário de Brasília)
"commit.committer": "luisciaramicoli" "2026-09-30T16:20:05Z" -> 30/09/2026, 13:20:05 (horário de Brasília)
"commit.message": "upload implementado"

```

Figura 12: Atividade 6 — commit ffa84d9 no GitHub — página do commit e data/hora exata pela API do GitHub

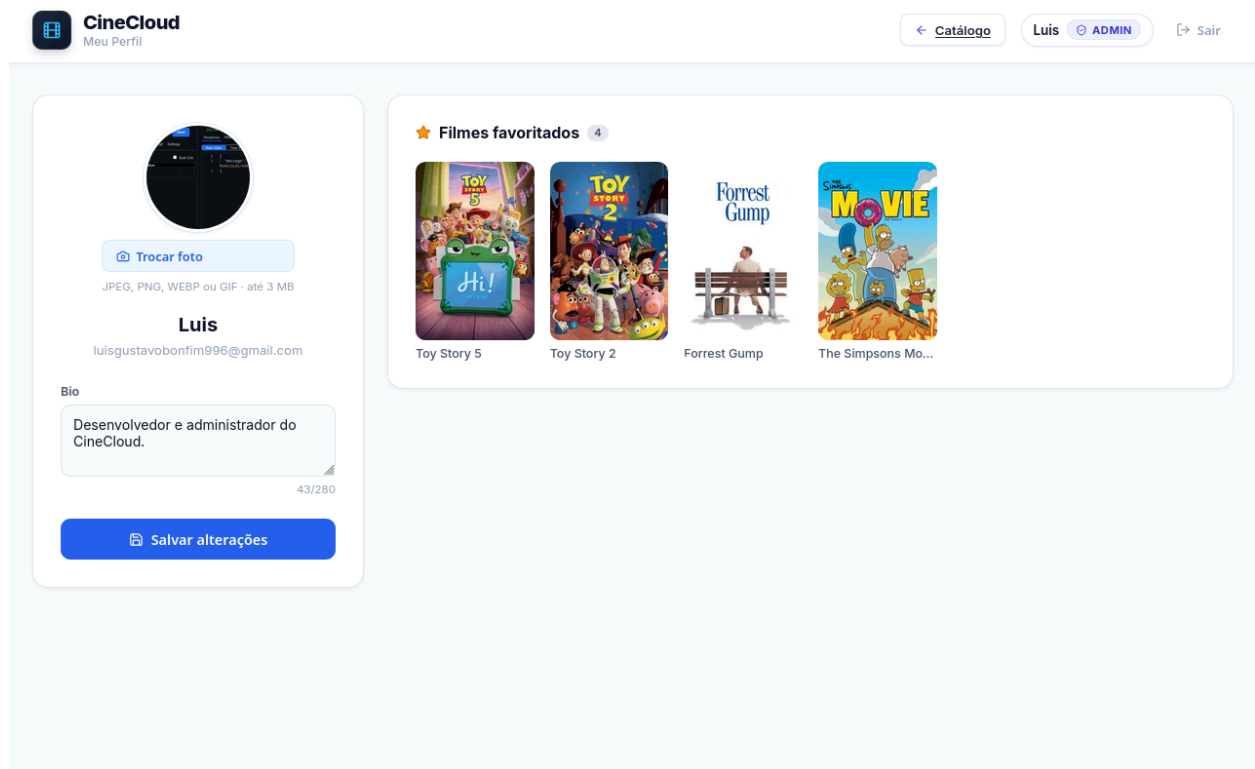


Figura 13: Atividade 6 — upload de avatar realizado com sucesso e refletido no perfil do usuário

Dificuldades e como foram resolvidas. Como o servidor do Portainer na Fatec utiliza uma porta única externa (8218), o navegador não conseguia acessar a porta padrão do MinIO (9000) para carregar a foto do avatar. Resolvi criando uma rota interna no backend (GET /api/avatar/:userId) que atua como proxy reverso: ela lê o stream do arquivo no MinIO pela rede interna do Docker e o transmite ao navegador com os cabeçalhos de imagem e cache apropriados.

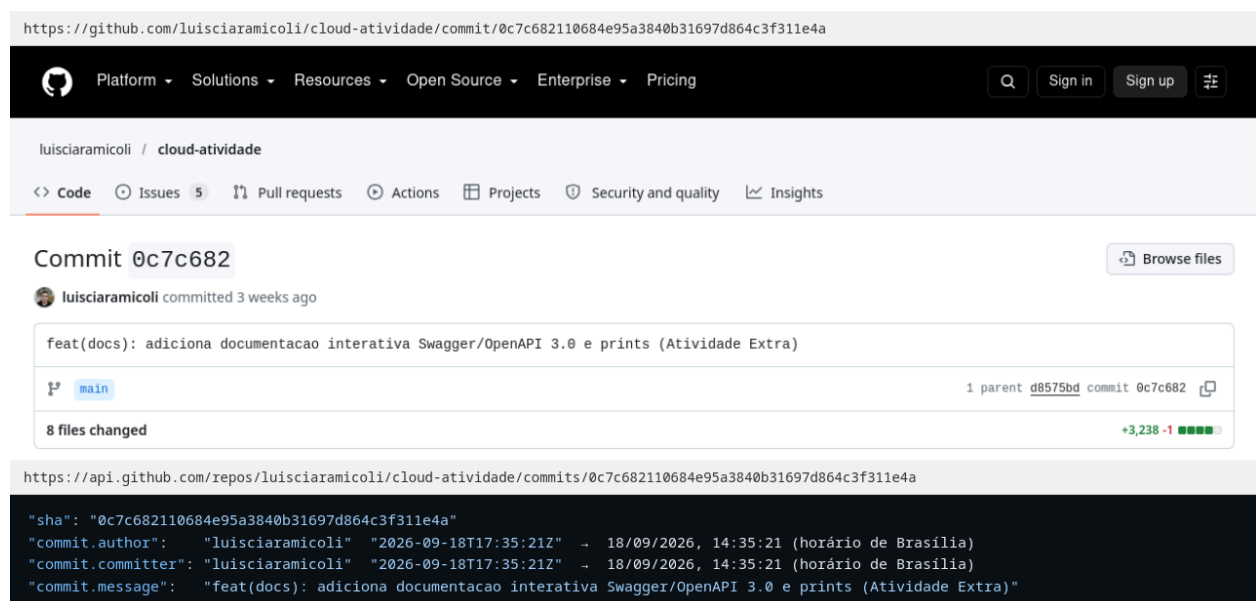
5 Atividades extras (opcional)

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.
Data planejada	sem prazo
Data realizada	18/09/2026 14:34
Situação	entregue
Evidência	GitHub — commit 1618dd2 + print do Swagger UI no navegador
Link	https://github.com/luiscaramicoli/cloud-atividade/commit/1618dd21d8b74a382d5fa4ae1a5477026e695503

O que foi feito. Documentei todos os endpoints da aplicação na especificação OpenAPI 3.0, cobrindo o backend do catálogo, o auth-service e o log-service. Integrei o swagger-ui-express para servir a interface gráfica interativa do Swagger na rota /api-docs. A documentação descreve todos os parâmetros, formatos das requisições, respostas de sucesso, mensagens de erro e o esquema de autenticação por cabeçalho Bearer Token.

<https://github.com/luiscaramicoli/cloud-atividade/commit/0c7c682110684e95a3840b31697d864c3f311e4a>



```

"sha": "0c7c682110684e95a3840b31697d864c3f311e4a"
"commit.author": "luiscaramicoli" "2026-09-18T17:35:21Z" -> 18/09/2026, 14:35:21 (horário de Brasília)
"commit.committer": "luiscaramicoli" "2026-09-18T17:35:21Z" -> 18/09/2026, 14:35:21 (horário de Brasília)
"commit.message": "feat(docs): adiciona documentacao interativa Swagger/OpenAPI 3.0 e prints (Atividade Extra)"

```

Figura 14: Extra E1 — commit 1618dd2 no GitHub — página do commit e data/hora exata pela API do GitHub

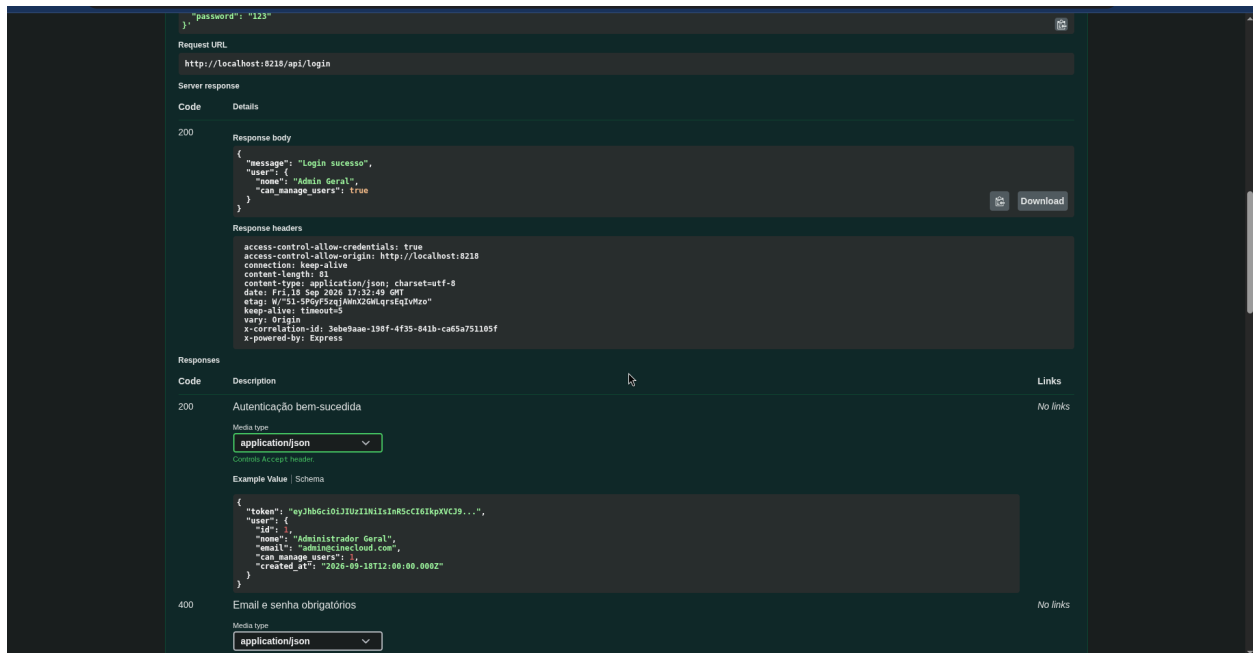


Figura 15: Extra E1 — interface interativa do Swagger UI permitindo testes dos endpoints via Try it out

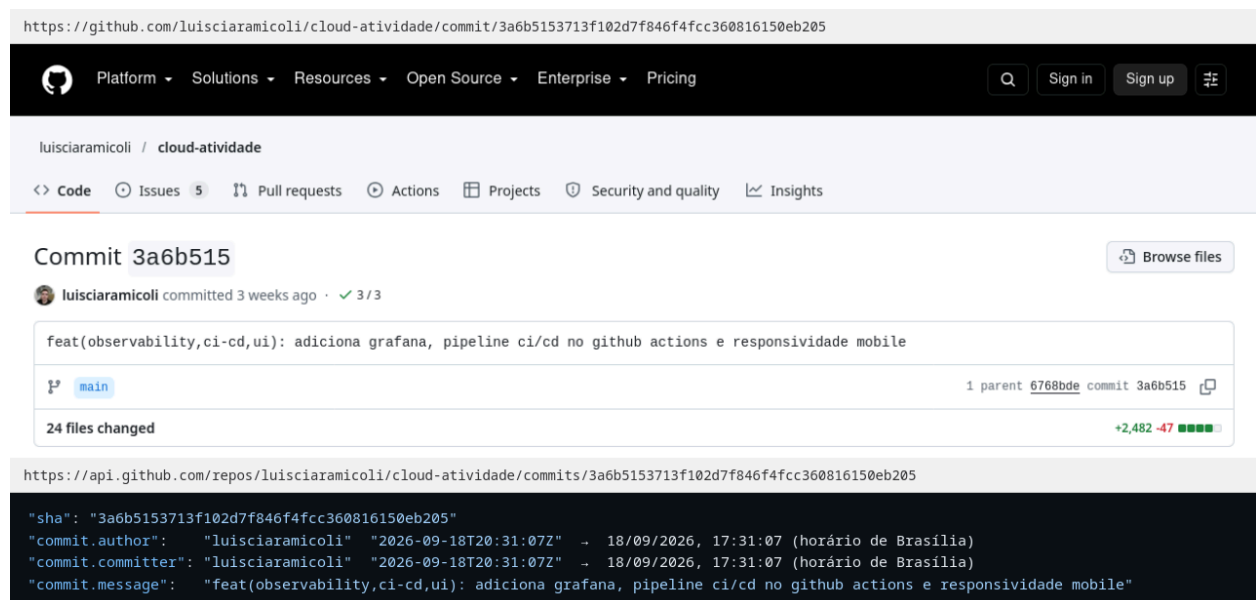
Dificuldades e como foram resolvidas. Como a aplicação estava dividida em múltiplos serviços, manter arquivos de especificação separados complicaria o uso pelos desenvolvedores. Resolvi unificando as definições em um único catálogo no gateway, facilitando a navegação de todas as rotas em um só lugar.

5.2 Atividade E2 — CI/CD com GitHub Actions

Descrição	Pipeline que testa e faz deploy a cada push.
Data planejada	sem prazo
Data realizada	18/09/2026 17:31
Situação	entregue
Evidência	GitHub — commit 3a6b515 + print do GitHub Actions executado
Link	https://github.com/luiscaramicoli/cloud-atividade/commit/3a6b5153713f102d7f846f4fcc360816150eb205

O que foi feito. Configurei uma esteira de Integração e Entrega Contínuas (CI/CD) com o GitHub Actions (.github/workflows/ci-cd.yml). A cada push no branch principal (main), o workflow executa a verificação de código, compila as imagens Docker dos microsserviços, publica os pacotes no GitHub Container Registry (GHCN) e dispara um webhook para o Portainer da faculdade, atualizando a stack de forma automática.

<https://github.com/luiscaramicoli/cloud-atividade/commit/3a6b5153713f102d7f846f4fcc360816150eb205>



Commit 3a6b515

luiscaramicoli committed 3 weeks ago · 3 / 3

feat(observability,ci-cd,ui): adiciona grafana, pipeline ci/cd no github actions e responsividade mobile

1 parent 6768bde commit 3a6b515

24 files changed +2,482 -47

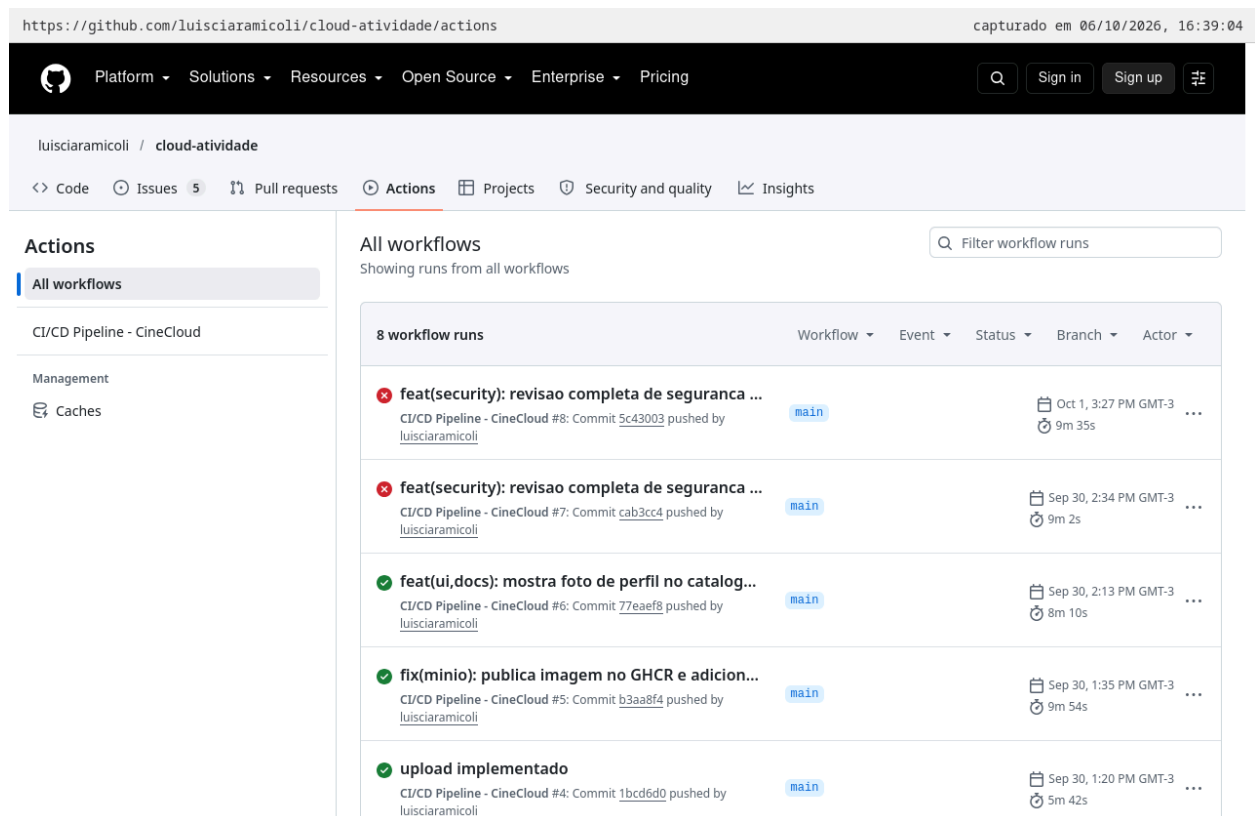
<https://api.github.com/repos/luiscaramicoli/cloud-atividade/commits/3a6b5153713f102d7f846f4fcc360816150eb205>

```

"sha": "3a6b5153713f102d7f846f4fcc360816150eb205"
"commit.author": "luiscaramicoli" "2026-09-18T20:31:07Z" → 18/09/2026, 17:31:07 (horário de Brasília)
"commit.committer": "luiscaramicoli" "2026-09-18T20:31:07Z" → 18/09/2026, 17:31:07 (horário de Brasília)
"commit.message": "feat(observability,ci-cd,ui): adiciona grafana, pipeline ci/cd no github actions e responsividade mobile"

```

Figura 16: Extra E2 — commit 3a6b515 no GitHub — página do commit e data/hora exata pela API do GitHub



https://github.com/luisciaramicoli/cloud-atividade/actions

capturado em 06/10/2026, 16:39:04

Platform Solutions Resources Open Source Enterprise Pricing

luisciaramicoli / cloud-atividade

<> Code Issues 5 Pull requests Actions Projects Security and quality Insights

Actions

All workflows

CI/CD Pipeline - CineCloud

Management

Caches

All workflows

Showing runs from all workflows

Filter workflow runs

8 workflow runs

	Workflow	Event	Status	Branch	Actor
✖ feat(security): revisao completa de seguranca ...	CI/CD Pipeline - CineCloud #8: Commit 5c43003 pushed by luisciaramicoli	main	Oct 1, 3:27 PM GMT-3	9m 35s	...
✖ feat(security): revisao completa de seguranca ...	CI/CD Pipeline - CineCloud #7: Commit cab3cc4 pushed by luisciaramicoli	main	Sep 30, 2:34 PM GMT-3	9m 2s	...
✔ feat(ui,docs): mostra foto de perfil no catalog...	CI/CD Pipeline - CineCloud #6: Commit 77eae8 pushed by luisciaramicoli	main	Sep 30, 2:13 PM GMT-3	8m 10s	...
✔ fix(minio): publica imagem no GHCR e adicion...	CI/CD Pipeline - CineCloud #5: Commit b3aa8f4 pushed by luisciaramicoli	main	Sep 30, 1:35 PM GMT-3	9m 54s	...
✔ upload implementado	CI/CD Pipeline - CineCloud #4: Commit 1bcd6d0 pushed by luisciaramicoli	main	Sep 30, 1:20 PM GMT-3	5m 42s	...

Figura 17: Extra E2 — pipeline do GitHub Actions executada com sucesso e publicação das imagens

Dificuldades e como foram resolvidas. O Portainer da Fatec não permite montagem direta de pastas do host (bind-mounts) para usuários sem privilégios de administrador. Para contornar essa restrição, alterei os Dockerfiles para copiar os arquivos de configuração estáticos diretamente para dentro das camadas das imagens (0d2c8da).

5.3 Atividade E3 — Observabilidade — health checks e métricas

Descrição	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.
Data planejada	sem prazo
Data realizada	18/09/2026 17:31
Situação	entregue
Evidência	GitHub — commit 3a6b515 + print do Grafana e endpoint de healthcheck
Link	https://github.com/luiscaramicoli/cloud-atividade/commit/3a6b5153713f102d7f846f4fcc360816150eb205

O que foi feito. Implementei a observabilidade do sistema com endpoints de saúde (/health, incluindo liveness e readiness) e exportação de métricas (/metrics) no formato Prometheus nos microserviços. O docker-compose.yml foi configurado com verificações nativas de healthcheck. Uma instância do Prometheus faz a coleta periódica dos dados e um dashboard no Grafana exibe tempo de atividade (uptime), requisições por minuto e latência p95. Para testar a tolerância a falhas, parei o container do Redis e confirmei que o log-service passou ao estado unhealthy, recuperando-se sozinho assim que o Redis voltou a operar.

<https://github.com/luiscaramicoli/cloud-atividade/commit/3a6b5153713f102d7f846f4fcc360816150eb205>

```

"sha": "3a6b5153713f102d7f846f4fcc360816150eb205"
"commit.author": "luiscaramicoli" "2026-09-18T20:31:07Z" -> 18/09/2026, 17:31:07 (horário de Brasília)
"commit.committer": "luiscaramicoli" "2026-09-18T20:31:07Z" -> 18/09/2026, 17:31:07 (horário de Brasília)
"commit.message": "feat(observability,ci-cd,ui): adiciona grafana, pipeline ci/cd no github actions e responsividade mobile"

```

Figura 18: Extra E3 — commit 3a6b515 no GitHub — página do commit e data/hora exata pela API do GitHub

```
http://localhost:8218/health
```

capturado em 06/10/2026, 16:38:56

```
{"status": "healthy", "service": "catalog-service", "checks": {"database": "connected"}, "timestamp": "2026-10-06T19:38:53.772Z"}
```

Figura 19: Extra E3 — endpoint de verificação de integridade (/health) reportando status healthy e conexão do banco

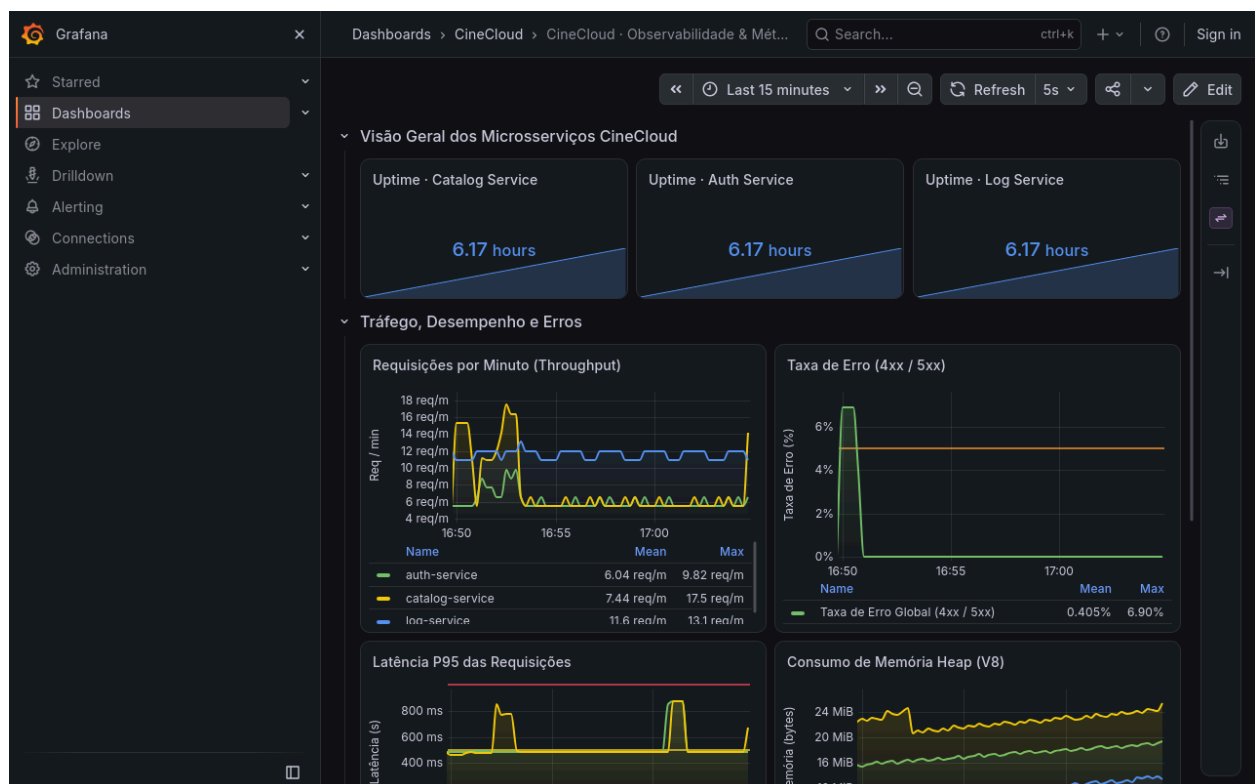


Figura 20: Extra E3 — dashboard executivo no Grafana monitorando uptime, tráfego e latência dos serviços

Dificuldades e como foram resolvidas. Como apenas a porta 8218 estava liberada para o meu usuário no servidor compartilhado da faculdade, não era possível expor a porta 3000 do Grafana diretamente. Resolvi configurando o Grafana com sub-caminho (serve_sub_path) e criando um proxy reverso no backend do catálogo para direcionar as requisições em /grafana diretamente ao container interno (066320b).

6 Considerações finais

O desenvolvimento deste projeto no primeiro bimestre permitiu entender na prática como funciona a transição de um sistema monolítico para uma arquitetura orientada a microsserviços. Ver na prática o desacoplamento, isolando a autenticação no auth-service, salvando arquivos no MinIO e despachando logs de auditoria no Redis Streams, mostrou as vantagens de manutenção e organização de um sistema em nuvem, mas também evidenciou os cuidados necessários com redes internas, comunicação entre serviços e tratamento de erros.

O maior desafio esteve nas limitações do ambiente compartilhado no Portainer da faculdade. Questões como a restrição de volumes por bind-mount, a liberação de uma única porta de acesso e o timeout na compilação do MinIO exigiram soluções práticas, como a criação de proxies reversos para centralizar rotas e a automação do build de imagens com GitHub Actions e GHCR.

Como melhoria para os próximos trabalhos e para o segundo bimestre, pretendo implementar testes de integração automatizados antes de cada commit e adotar o padrão de Conventional Commits desde o primeiro dia de versionamento. A base construída até aqui ficou modular, documentada e estável, pronta para receber os temas seguintes da disciplina, como o Plano Premium e os modelos de serviço IaaS, PaaS e SaaS.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Documento assinado digitalmente
 LUIS GUSTAVO BONFIM CIARAMICOLI
Data: 07/10/2026 17:27:56-0300
Verifique em <https://validar.iti.gov.br>

Luis Gustavo Bonfim Ciaramicoli

Pompeia, 07/10/2026